Practical - 6

Rendering External JSON Data using Fetch API, Search and Filter

Display event lists, student profiles, notices, or FAQs from external JSON files using Fetch API. Implement dynamic rendering, search, filter, sorting, and pagination.

A. Theory

1. How is JSON fetched, parsed, and rendered?

Answer :

It uses the modern fetch("../DATA/events.json") API. It is parsed by returning response.json() in the first .then() block. To render, it loops through the parsed array using .forEach(), creates new div elements with document.createElement("div"), populates their innerHTML with template literals, and appends them to the #eventsList container.

2. Which array methods are used for search, filter, sort, and pagination?

- Search & Filter: The .filter() method is used to return a new array containing only events that match the text input and selected dropdown category.
- Sort: The .sort() method is used to reorder dates by comparing their timestamp values (getTime()).
- Pagination: The .slice() method is used to extract a specific chunk of the array (e.g., index 0 to 5) to display only the items for the currentPage.

3. How are loading and error states handled?

Answer :

A loading state is hardcoded into the HTML initially (Loading events...). If the fetch() request fails (network error or missing file), a .catch() block catches the error and updates the DOM to display a red error message: "Failed to load events. Please try again later."

4. How is modularity maintained in JavaScript files?

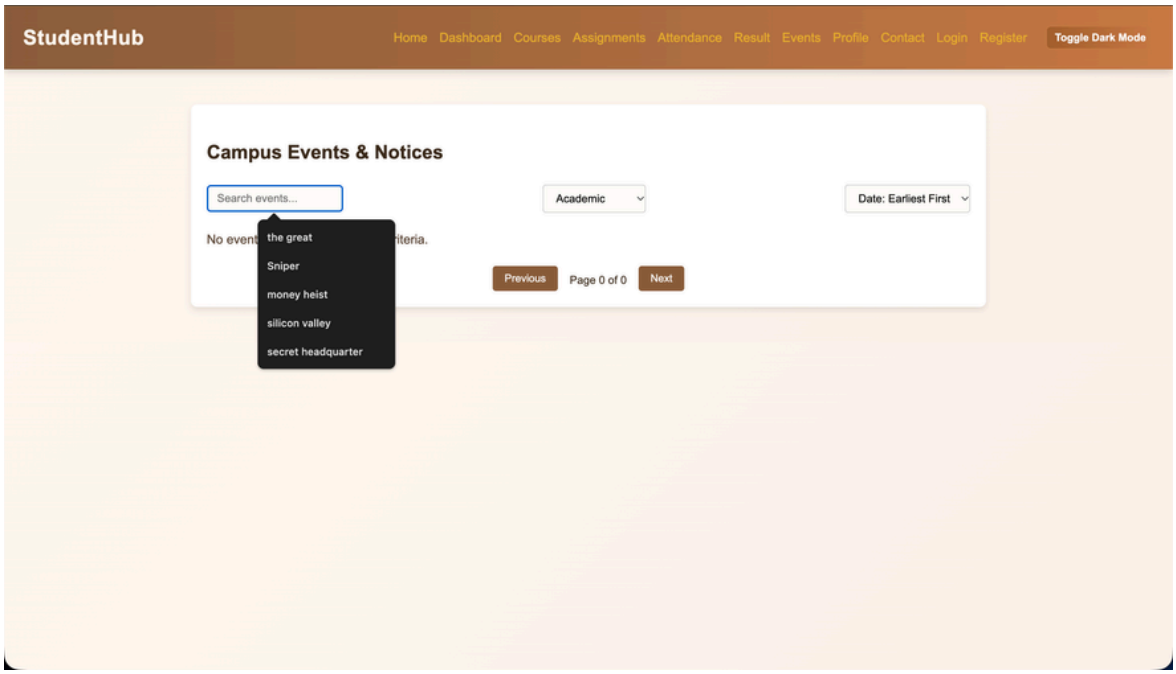
Answer :

Modularity is achieved by separating the heavy event-fetching logic into a completely separate file (JS/events.js), keeping it isolated from the global portal functions (like Dark Mode) which remain in java.js.

B. Implementation

Expanded the portal by adding a new events.html page and updating the navigation menus across all existing pages to include it. Furthermore, a Pagination UI component was implemented in script.css using CSS Flexbox to allow users to navigate through long lists (like events or courses).

C. Evidence of Work (Screenshots)



D. GitHub Repository Link:

https://github.com/palghori/STUDENT-HUB-PORTAL-WDF/tree/main/practical%206

E. Challenges Faced:

- Keeping the navigation bar synchronized across 10+ separate HTML files without a backend framework (like React or PHP includes).
- Getting the pagination buttons (Previous/Next) to align perfectly and handling their visual states (e.g., styling :disabled buttons so they look unclickable).

F. Learning Outcomes:

Improved CSS Layout skills (specifically Flexbox for the pagination controls), understanding UI component states, and experiencing firsthand the scaling limitations of purely static HTML files.

Faculty Remarks: